Near-linear pseudo-polynomial algorithms for minimizing tardy processing time on one machine

Nick Fischer, Leo Wennmann

arxiv:2402.13357v3; CC BY 4.0

Editorial note. Only theo:mtpt counts, jointly including its randomized and deterministic algorithms. Full author processing-time model, pseudo-polynomial parameter P and proofs retained; publisher class/metadata is adapted to article and original paper.bib rendered via installed natbib/BibTeX. Native source archive preserved.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

In this work we revisit the elementary scheduling problem $1||\sum p_jU_j$. The goal is to select, among n jobs with processing times and due dates, a subset of jobs with maximum total processing time that can be scheduled in sequence without violating their due dates. This problem is NP-hard, but a classical algorithm by Lawler and Moore from the 60s solves this problem in pseudo-polynomial time $O(nP)$, where P is the total processing time of all jobs. With the aim to develop best-possible pseudo-polynomial-time algorithms, a recent wave of results has improved Lawler and Moore's algorithm for $1||\sum p_jU_j$: First to time $\widetilde{O}(P^{7/4})$ [Bringmann, Fischer, Hermelin, Shabtay, Wellnitz; ICALP'20], then to time $\widetilde{O}(P^{5/3})$ [Klein, Polak, Rohwedder; SODA'23], and finally to time $\widetilde{O}(P^{7/5})$ [Schieber, Sitaraman; WADS'23]. It remained an exciting open question whether these works can be improved further.

In this work we develop an algorithm in near-linear time $\widetilde{O}(P)$ for the $1||\sum p_jU_j$ problem. This running time not only significantly improves upon the previous results, but also matches conditional lower bounds based on the Strong Exponential Time Hypothesis or the Set Cover Hypothesis and is therefore likely optimal (up to subpolynomial factors). Our new algorithm also extends to the case of m machines in time $\widetilde{O}(P^m)$. In contrast to the previous improvements, we take a different, more direct approach inspired by the recent reductions from Modular Subset Sum to dynamic string problems. We thereby arrive at a satisfyingly simple algorithm.

1 Introduction

Consider the following natural optimization problem: A worker is offered n jobs, where each job j requires a processing time of p_j days and must be completed before some due date d_j. Which jobs should the worker take on in order to maximize their pay, assuming that the worker is paid a fixed amount per day of work? In standard scheduling notation [22], this task is somewhat cryptically called the "$1||\sum p_jU_j$" problem (see Section 2 for a formal definition). The $1||\sum p_jU_j$ problem constitutes an important scheduling task that is arguably among the simplest nontrivial scheduling objectives, and has received considerable attention in the literature, especially in recent years.

The $1||\sum p_jU_j$ problem naturally generalizes the famous Subset Sum problem,[1] and is therefore NP-hard. However, it does admit pseudo-polynomial-time algorithms—in 1969, Lawler and Moore [31] pioneered the first such algorithm in time $O(nP)$, where $P = \sum_j p_j$ is the total processing time of all n jobs. This result is the baseline in a line of research that, more than 50 years after the initial effort, is finally brought to a close in this paper.

State of the Art. Lawler and Moore originally designed their algorithm for a weighted generalization of the $1||\sum p_jU_j$ problem, and for this generalization the running time $O(nP)$ was proven to be conditionally tight.[2] Even for the $1||\sum p_jU_j$ problem the Lawler-Moore algorithm remained unchallenged for a long time. Only a few years ago, Bringmann, Fischer, Hermelin, Shabtay and Wellnitz [13] managed to solve $1||\sum p_jU_j$ in time[3] $\widetilde{O}(P^{7/4})$, showcasing that improvements over Lawler-Moore are indeed possible in certain parameter regimes (specifically, when $P \ll n^{4/3}$). Their strategy is to design a reduction to an intermediate problem called Skewed Convolution[4], and to develop an $\widetilde{O}(N^{7/4})$-time algorithm for this intermediate problem.

Their work was later improved in two orthogonal ways. On the one hand, Klein, Polak and Rohwedder [28] improved the running time of Skewed Convolution to $\widetilde{O}(N^{5/3})$. On the other hand, Schieber and Sitaraman [36] improved the algorithmic reduction and established that, if Skewed Convolution is in time $\widetilde{O}(N^{\alpha})$, then $1||\sum p_jU_j$ is in time $\widetilde{O}(P^{2-1/\alpha})$. The state-of-the-art algorithm for $1||\sum p_jU_j$ is obtained by combining these two works, resulting in time $\widetilde{O}(P^{7/5})$.

In contrast, fine-grained lower bounds for the Subset Sum problem rule out $1||\sum p_jU_j$ algorithms in time $O(P^{1-\epsilon} \cdot n^{O(1)})$, for any $\epsilon > 0$, conditioned on either the influential Strong Exponential Time Hypothesis [1] or the Set Cover Hypothesis [17]. This leaves a substantial gap between the best known upper bound $\widetilde{O}(P^{7/5})$ and the conceivable optimum $\widetilde{O}(P)$. Closing this gap is the starting point of our paper:

[1] Indeed, Subset Sum is the special case of $1||\sum p_jU_j$ where all jobs share the same deadline d. In other words, Subset Sum is the $1|d_j = d|\sum p_jU_j$ problem.

[2] In the so-called $1||\sum w_jU_j$ problem each job j is rewarded by a specified pay w_j (instead of p_j). For this generalization the running time $O(nP)$ was proven to be conditionally optimal [18, 30], in the sense that an algorithm in time $O((n+P)^{2-\epsilon})$, for any $\epsilon > 0$, contradicts the well-established $(\min, +)$-Convolution hypothesis. See also the discussion in [13].

[3] We write $\widetilde{O}(T) = T(\log T)^{O(1)}$ to suppress polylogarithmic factors.

[4] Given length-N integer vectors A, B, the Skewed Convolution problem is to compute the length-$(2N-1)$ vector C defined by $C[k] = \min_{i+j=k} \max\{A[i], B[j] - i\}$.

Can the $1||\sum p_jU_j$ problem be solved in near-linear time $\widetilde{O}(P)$?

In light of the recent algorithmic developments [13, 28, 36], a reasonable strategy appears to aim for even faster algorithms for the Skewed Convolution problem—unfortunately, this approach soon faces a barrier. Namely, improving the running time of Skewed Convolution beyond $O(N^{3/2})$ would entail a similarly fast algorithm for $(\max, \min)$-Convolution, which, while not ruled out under one of the big hypotheses, would be a surprising break-through in fine-grained complexity theory. This leaves us in an uncertain situation. Even if Skewed Convolution could be improved to time $\widetilde{O}(N^{3/2})$, this would mean that the $1||\sum p_jU_j$ problem is in time $\widetilde{O}(P^{4/3})$ [36]. Are further improvements impossible?

Our Results. In this paper we bypass this barrier and develop a new algorithm for $1||\sum p_jU_j$ that avoids the reduction to Skewed Convolution altogether. We thereby successfully resolve our driving question:

Theorem 1.1. The $1||\sum p_jU_j$ problem can be solved in randomized time $O(P \log P)$ and in deterministic time $O(P \log^{1+o(1)} P)$.

We stress that by the aforementioned lower bounds [1, 17] our new algorithm is optimal, up to lower-order factors, conditioned on the Strong Exponential Time Hypothesis or the Set Cover Hypothesis.

As an additional feature, and similar to all previous algorithms, our algorithm not only computes the optimal value of the given instance, but in fact reports for each value $0 \leq x \leq P$ whether there is a feasible schedule with processing time (i.e., pay) x. Moreover, we can compute an optimal schedule (represented as an ordered subset of jobs) in the same running time.

Another benefit of our work is that we managed to distill an astonishingly simple algorithm. In fact, our algorithm is basically identical to the original Lawler-Moore algorithm, except that we replace certain naive computations by an appropriate efficient data structure on strings, and employ a careful new analysis. This approach is inspired by the recent progress on the Modular Subset Sum problem [5, 6, 14, 34] (see Section 3 for more details). We find it surprising that these conceptually simple ideas lead to near-optimal running times for $1||\sum p_jU_j$.

In particular, in contrast to previous improvements for $1||\sum p_jU_j$ [13, 28, 36], our algorithm is purely combinatorial and does not require the use of the Fast Fourier Transform. Given this simple nature of our algorithm, we are confident that actual implementations of the algorithm would perform well.

Multiple Machines. The "$P_m||\sum p_jU_j$" problem is the straightforward generalization of the $1||\sum p_jU_j$ problem to m workers (or machines) that can partition the jobs arbitrarily among themselves. The goal, as before, is to maximize the total workload across all workers while respecting all due dates. We assume for simplicity that m is a constant.[5]

The Lawler-Moore algorithm generalizes in a straightforward manner to time $O(nP^m)$. For the algorithms based on Skewed Convolution, it seems significantly harder to derive multiple-machine generalizations. Luckily, with some appropriate changes our new algorithm also generalizes to multiple machines:

[5]When viewing m as an input, it is easy to trace that our algorithms depend only polynomially on m.

Theorem 1.2. The $P_m||\sum p_jU_j$ problem can be solved in randomized time $O(P^m \log P)$ and in deterministic time $O(P^m \log^{1+o(1)} P)$.

In particular, Theorem 1.2 outperforms the Lawler-Moore algorithm by a near-linear factor $\widetilde{\Omega}(n)$. In contrast to the single-machine setting, however, we emphasize that this algorithm is not necessarily optimal. A conditional lower bound for this problem would, most likely, be derived from an analogous lower bound for the multiple-target Subset Sum problem [3]. This appears to be a challenging question which is not resolved yet.

Alternative Parameters. So far we have only mentioned the parameters n and $P = \sum_j p_j$, which have been the main focus in previous work. But there are many other parameters worth considering. Natural candidates include the number of distinct deadlines ($D_\#$), the sum of all distinct deadlines (D), the largest processing time ($p_{\max} = \max_j p_j$) and the largest deadline ($d_{\max} = \max_j d_j$).

There has been research on developing nontrivial $1||\sum p_jU_j$ algorithms (for a single machine) with respect to these parameters, such as an $\widetilde{O}(\min\{P \cdot D_\#, P + D\})$-time algorithm due to [13], and an $\widetilde{O}(n + p_{\max}^3)$-time algorithm due to [28]. We remark that the former is subsumed by our new results. The latter algorithm is incomparable to our result (specifically, our algorithm in time $\widetilde{O}(P) = \widetilde{O}(np_{\max})$ performs better if and only if $p_{\max} \gg n^{1/2}$). Both of these results [13, 28] generalize to m machines as well, leading to similar comparisons with our work.

It remains an interesting open question whether our $\widetilde{O}(P)$-time algorithm can be further improved with respect to the parameters $d_{\max}$ and $p_{\max}$. The Lawler-Moore algorithm achieves a running time of $O(nd_{\max})$, but in principle it seems reasonable that time $\widetilde{O}(n + d_{\max})$ can be achieved, as the analogous question for Subset Sum is resolved [8, 26]. We leave this as an open question. An even more exciting question is whether we could possibly achieve time $\widetilde{O}(n + p_{\max})$. However, such an algorithm would entail a break-through for Subset Sum with small items, which currently seems out of reach.

Further Related Work. This work is part of a bigger effort of the fine-grained complexity community to design best-possible pseudo-polynomial time algorithms for a host of optimization problems. This line of research encompasses, besides the aforementioned scheduling problems [13, 23, 28, 36], various variants of Subset Sum [29, 8, 1, 5, 4, 12, 33, 6, 14, 34, 19], Knapsack [38, 18, 30, 7, 20, 33, 11, 16, 10, 9, 25], Integer Programming [20, 24] and many others [15, 19].

2 Preliminaries

Throughout, we write $[n] = \{0, \ldots, n-1\}$ and use the interval notation $[i \,..\, j] = \{i, \ldots, j\}$, and similarly $[i \,..\, j)$, $(i \,..\, j]$, $(i \,..\, j)$. For two sets of integers S, T and an integer t we employ the sumset notation $S + t = \{s + t : s \in S\}$ and $S + T = \{s + t : s \in S, t \in T\}$.

Scheduling Problems. We begin with a formal definition of the $1||\sum p_jU_j$ problem. The input consists of n jobs, where each job $j \in [n]$ has a pro-

cessing time $p_j \in \mathbb{N}_{>0}$ and due dates $d_j \in \mathbb{N}_{>0}$. A schedule is a permutation $\sigma : [n] \to [n]$. The completion time C_j of a job j in the schedule σ is $C_j = \sum_{i:\sigma(i)\leq\sigma(j)} p_i$ (i.e., the total processing time of all jobs preceding j, including j itself). We say that j is early if $C_j \leq d_j$ and tardy otherwise, and let $U_j \in \{0, 1\}$ be the indicator variable indicating whether j is tardy. In this notation, our objective is to find a schedule minimizing $\sum_j p_j U_j$ (i.e., the total processing time of all tardy jobs). This explains the description $1||\sum p_j U_j$ in three-field notation.[6] For convenience we have defined the problem in such a way that $p_j > 0$, and as a consequence we can always bound $n \leq P$.[7]

For the m-machine problem $P_m||\sum p_j U_j$ a schedule is analogously defined as a function $\sigma : [n] \to [n] \times [m]$, where the first coordinate determines the order of the jobs as before, and the second coordinate determines the machine which is supposed to execute the job. The completion time C_j is the total processing time of all jobs preceding j on j's machine (including j itself), and the objective of the problem remains unchanged. For simplicity, we assume throughout the paper that m is a constant (it can easily be verified that we only omit $m^{O(1)}$-factors this way).

Earliest-Due-Date-First Schedules. A key observation about $1||\sum p_j U_j$ dating back to Lawler and Moore [31] is that, without loss of generality, the early jobs are scheduled in increasing order of their due dates. This observation is leveraged as follows: We reorder the jobs such that $d_0 \leq \cdots \leq d_{n-1}$ (we will stick to this ordering for the rest of the paper). Thus, the $1||\sum p_j U_j$ problem is effectively to compute a subset of jobs $J \subseteq [n]$ that maximizes $\sum_{j\in J} p_j$ and is feasible in the sense that all jobs in J are early (i.e., $C_j = \sum_{i\in J:i\leq j} p_i \leq d_j$ for all $j \in J$).

Machine Model. We work in the standard Word RAM model with word size $\Theta(\log n + \log P)$ (such that each job can be stored in a constant number of cells). Moreover, all randomized algorithms mentioned throughout are Las Vegas (i.e., zero-error) algorithms running in their claimed time bounds with high probability $1 - 1/n^{\Omega(1)}$.

3 Near-Optimal Algorithm for a Single Machine

In this section, we give the details of our near-optimal algorithm for $1||\sum p_j U_j$. We start with a brief summary of the Lawler-Moore algorithm.

Lawler and Moore's Baseline. The Lawler-Moore algorithm [31] is the natural dynamic programming solution for the $1||\sum p_j U_j$ problem. We present it

[6] The 1 in the first field denotes a single machine, the empty second field symbolizes no additional constraints, and the third field gives the objective to minimize $\sum_j p_j U_j$.

[7] If jobs with processing time $p_j = 0$ were permitted, all of our algorithms would additionally require $O(n)$ time preprocessing.

here by recursively defining the following sets $S_0, \dots, S_n \subseteq [0\,..\,P]$:

$$\begin{aligned} S_0 &= \{0\}, \\ S'_{j+1} &= S_j + \{0, p_j\} && (j \in [n]), \\ S_{j+1} &= S'_{j+1} \cap [0, d_j] && (j \in [n]). \end{aligned}$$

(The construction of S_{j+1} is divided into two steps as this will be convenient later on.) Each set S_{j+1} can naively be computed from S_j in time $O(P)$, and thus all sets $S_0, \dots, S_n$ can be naively computed in time $O(nP)$. We can ultimately read off the optimal value as $\max S_n$, based on the following observation:

Observation 3.1 (Lawler and Moore [31]). There is a feasible schedule of total processing time t if and only if $t \in S_n$.

More generally, S_{j+1} is the set of processing times of feasible schedules involving the jobs $0, \dots, j$. To see this, consider any feasible schedule of the jobs $0, \dots, j-1$ (whose processing time is in S_j). We can either leave out the next job j or append to the schedule. Thereby, the set of processing times becomes $S'_{j+1} = S_j + \{0, p_j\} = \{s, s + p_j : s \in S'_j\}$. However, this appended schedule is not necessarily feasible as it might not comply with the due date d_j. Hence, all processing times greater than d_j are deleted again in the construction of S_{j+1}.

Our Approach. Perhaps surprisingly, our algorithm essentially follows the same approach, i.e., our goal remains to compute the sets $S_0, \dots, S_n$. However, we will demonstrate that the naive $O(P)$-time computation of each step can be significantly sped up. Our algorithm relies on two ingredients—an algorithmic and a structural one.

Ingredient 1: An Efficient Data Structure. As the sets S'_j and S_j are constructed in a highly structured way, can we compute them faster than time $O(P)$? Specifically, is there a way to (i) compute each set S'_{j+1} in time proportional to the number of insertions $|S'_{j+1} \setminus S_j|$, and to (ii) compute S_{j+1} in time proportional to the number of deletions $|S'_{j+1} \setminus S_{j+1}|$?

Question (i) is closely related to the Subset Sum problem, and has been successfully resolved in [5] leading to near-optimal algorithms for Modular Subset Sum. Their solution based on linear sketching is quite involved [5], but two independent papers [6, 14] provided a significantly simpler proof by replacing linear sketching with a reduction to a dynamic string problem; see also [34][8]. Regarding (ii), it turns out that we can adapt this reduction to the dynamic string problem to efficiently accommodate our deletions. The following lemma summarizes the resulting data structure; we defer its proof to Section 3.1.

Lemma 3.1 (Sum-Cap Data Structure). There is a randomized data structure that maintains a set $S \subseteq [u]$ and supports the following operations:

- `init`(S): Initializes the data structure to the given set $S \subseteq [u]$. Runs in time $O(|S| \cdot \log u + \log^2 u)$.

[8] In [34], Potępa proposes an improved deterministic data structure with applications to the Modular Subset Sum problem. A priori, it looks like their improvement might similarly apply to our setting. Unfortunately, the data structure is only efficient if we have the freedom to arbitrarily reorder the items, which is prohibitive for us as we have to stick to the order $d_0 \leq \cdots \leq d_{n-1}$.

- `query`(s): Given $s \in [u]$, tests whether $s \in S$.
  Runs in time $O(\log u)$.
- `sum`(p): Given $p \in [u]$, updates $S \leftarrow S + \{0, p\}$.
  Runs in time $O(|(S+p) \setminus S| \cdot \log u)$ (where S is as before the operation).
- `cap`(d): Given $d \in [u]$, updates $S \leftarrow S \cap [d]$.
  Runs in time $O(\log u)$.

If at any point during the execution an element $s \notin [u]$ is attempted to be inserted, the data structure becomes invalid. Moreover, the data structure can be made deterministic at the cost of worsening all operations by a factor $\log^{o(1)} u$.

Ingredient 2: A Structural Insight. What have we gained by computing the sets S_j and S'_j with the data structure from Lemma 3.1? Due to the particularly efficient `cap` operation, the computation of the sets S_j is essentially for free. Computing the sets S'_j using the `sum` operation, however, amounts to time

$$\widetilde{O}\left(\sum_{j\in[n]} |S'_{j+1} \setminus S_j|\right).$$

A priori, it is not clear whether this is helpful. In case of only inserting elements, this sum could be conveniently bounded by P (as is the case for Modular Subset Sum). Unfortunately, we additionally have to deal with deletions. Specifically, it is possible that some element s is inserted in S'_1, deleted in S_1, inserted again in S'_2, and so on. All in all, s could be inserted up to n times, and so the only immediate upper bound for the sum is nP (which recovers the Lawler-Moore running time).

Our crucial structural insight is that, while the same element can indeed be inserted and deleted multiple times, the total number of insertions is nevertheless bounded. More precisely, we show that the overall number of insertions is at most $O(P)$:

Lemma 3.2 (Bounded Insertions). It holds that $\sum_{j\in[n]} |S'_{j+1} \setminus S_j| \leq 2P + 1$.

Proof. We split the sum into two parts:

$$\sum_{j\in[n]} \big|(S'_{j+1} \setminus S_j)\big| = \sum_{j\in[n]} \big|(S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]\big| + \sum_{j\in[n]} \big|(S'_{j+1} \setminus S_j) \cap (d_j\,..\,P]\big|.$$

Intuitively, the first sum counts the number of elements that are irreversibly inserted into the sets $S_{j+1}, \ldots, S_n$ in the j-th step. The second sum counts the number of elements that are inserted into S'_{j+1} and immediately deleted in S_{j+1}.

For the first sum, consider the following observation: For any $x \in [0\,..\,P]$, if $x \leq d_j$ and $x \in S'_{j+1} \setminus S_j$, then $x \in S_{j+1}, \ldots, S_n$ (since $d_j \leq d_{j+1}, \ldots, d_{n-1}$). It follows that

$$\big|\{j \in [n] : x \in (S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]\}\big| \leq 1$$

for all $x \in [0\,..\,P]$. Thus,

$$\sum_{j\in[n]} \big|(S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]\big| = \sum_{x\in[0\,..\,P]} \big|\{j \in [n] : x \in (S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]\}\big| \leq P + 1.$$

Second, we bound $|(S'_{j+1} \setminus S_j) \cap (d_j\,..\,P]| \leq |S'_{j+1} \cap (d_j\,..\,P]|$. Note that $S_j \subseteq [0\,..\,d_{j-1}]$ and therefore $S'_{j+1} = S_j + \{0, p_j\} \subseteq [0\,..\,d_{j-1} + p_j]$. Consequently,

$$\big|S'_j \cap (d_j\,..\,P]\big| \leq \big|[0\,..\,d_{j-1} + p_j] \cap (d_j\,..\,P]\big| \leq d_{j-1} + p_j - d_j \leq p_j,$$

where the final inequality follows from the assumption that $d_{j-1} \leq d_j$. Hence, the number of overall deletions is

$$\sum_{j\in[n]} \big|(S'_{j+1} \setminus S_j) \cap (d_j\,..\,P]\big| \leq \sum_{j\in[n]} p_j = P.$$

Combining both parts concludes the proof. □

The proof for Lemma 3.1 is provided in Section 3.1. Using Lemma 3.2 and 3.1, we are in the position to show our main result.

Theorem 1.1. (Restated) The $1||\sum p_jU_j$ problem can be solved in randomized time $O(P \log P)$ and in deterministic time $O(P \log^{1+o(1)} P)$.

Proof. In summary, our algorithm works as follows. We compute $S_0, \ldots, S_n \subseteq [0\,..\,P]$ using the data structure from Lemma 3.1 (with $u = P + 1$). Specifically, after initializing S with $\texttt{init}(S_0)$, we repeatedly construct the sets S'_j and S_j using the operations $\texttt{sum}(p_j)$ and $\texttt{cap}(d_j)$ for all $j \leftarrow 0, \ldots, n-1$. The largest element in the final set $S = S_n$ is the maximal total processing of a feasible schedule of all jobs $0, \ldots, n-1$. Finding and returning it is the last step of our algorithm, by repeatedly using the $\texttt{query}(i)$ operation and returning the largest index i for which the query returns yes. The correctness of our algorithm follows from Observation 3.1.

The running time is composed of the following parts: The initialization runs in time $O(\log^2 P)$, the repeated use of `sum` and `cap` takes time $O(\sum_{j\in[n]}(|S'_{j+1} \setminus S_j| \cdot \log P + \log P))$ and the optimal value is found in time $O(P)$. Using $n \leq P$ and Lemma 3.2, it holds that

$$O\left(\sum_{j\in[n]} (|S'_{j+1} \setminus S_j| \cdot \log P + \log P)\right) \leq O(P \log P).$$

In total, we have a randomized running time of $O(P \log P)$. Applying the same arguments yields the deterministic running time of $O(P \log^{1+o(1)} P)$. □

We stress that the algorithm described in this section only computes the optimal value. In Section 3.2, we explain how our algorithm can be easily extended to obtain the optimal schedule as well.

3.1 Cap-Sum Data Structure via Dynamic Strings

In this section, we provide the missing proof of Lemma 3.1 by a reduction to the dynamic strings data structure problem. This is the fundamental problem of maintaining a collection of strings that can be concatenated, split, updated, and tested for equality—see [37, 35, 32, 2, 21]. We summarize the state of the art in the following lemma; the fastest randomized (and in fact, optimal) data structure is due to Gawrychowski, Karczmarz, Kociumaka, Lacki and Sankowski [21], and for the fastest deterministic one see [27, Section 8].

Here we use standard string notation for a string x, where $x[i]$ denotes the letter at index i, and $x[i\,..\,j], x[i\,..\,j)$ denote the appropriate substrings.

Lemma 3.3 (Dynamic String Data Structure [21, 27]). There is a data structure that maintains a dynamic collection X of non-empty strings and support the following operations:

- `make_string`(x): Given any string $x \in \Sigma^+$, inserts x into X.
- `concat`(x_1, x_2): Given $x_1, x_2 \in X$, inserts the concatenation x_1x_2 into X.
- `split`(x, i): Given $x \in X$ and $i \in [0\,..\,|x|)$, inserts $x[0\,..\,i]$ and $x(i\,..\,|x|)$ into X.
- `LCP`(x_1, x_2): Given $x_1, x_2 \in X$, returns the length ℓ of their longest common prefix, i.e., returns $\max\{0 \le \ell \le \min\{|x_1|, |x_2|\} : x_1[0\,..\,\ell) = x_2[0\,..\,\ell)\}$.
- `query`(x, i): Given $x \in X$ and $i \in [|x|]$, returns $x[i]$.

Let n be the maximum of the total length of all strings and the number of executed operations. Then all operations run in randomized time $O(\log n)$ or in deterministic time $O(\log^{1+o(1)} n)$, except for `make_string` which takes time $O(|x| + \log n)$ and $O(|x| \cdot \log^{o(1)} n)$, respectively.

For the sake of convenience, we include two more dynamic string operations that are derived from the previous lemma in a black-box fashion. As both are standard operations [21], we only provide their implementations for completeness.

- `update`(x, i, σ): Given $x \in X$, an index $i \in [|x|]$ and $\sigma \in \Sigma$, inserts the string obtained from x by changing the i-th character to σ into the data structure. To implement this, we split the string x twice to separate the letter $x[i]$ from the rest of the string. Specifically, we obtain the substring $x[0\,..\,i]$ using `split`(x, i) and further divide it to get the substring $x[0\,..\,i)$ by `split`$(x[0\,..\,i), i-1)$. Next, the `make_string`(σ) operation creates the string σ. Lastly, we use `concat`$(x[0\,..\,i), $`concat`$(\sigma, x(i\,..\,|x|)))$ to reinsert σ between the two substrings.
- `LCE`(x_1, x_2, i_1, i_2): Given $x_1, x_2 \in X$ and $i_1 \in [|x_1|], i_2 \in [|x_2|]$, returns the longest common extension $\max\{0 \le \ell \le \min\{|x_1| - i_1, |x_2| - i_2\}$:

$x_1[i_1 \,..\, i_1 + \ell) = x_2[i_2 \,..\, i_2 + \ell)\}$. To implement this, using the two operations $\texttt{split}(x_1, i-1)$ and $\texttt{split}(x_2, j-1)$ we first separate the substrings $x_1[i \,..\, |x_1|)$ and $x_2[j \,..\, |x_2|)$. Observe that the length of the longest common extension of the original strings is exactly the length of the longest common prefix of $x_1[i \,..\, |x_1|)$ and $x_2[j \,..\, |x_2|)$ returned by the operation $\texttt{LCP}(x_1[i \,..\, |x_1|), x_2[j \,..\, |x_2|))$.

Both `update` and `LCE` require a constant number of original operations that run in randomized time $O(\log n)$, or deterministic time $O(\log^{1+o(1)} n)$.

Now, we are in the position to provide the proof of Lemma 3.1. Recall that this proof is in parts borrowed from [6, 14].

Proof of Lemma 3.1. We maintain the set $S \subseteq [u]$ as an indicator string $x_S \in \{0,1\}^u$ such that $i \in S$ if and only if $x_S[i] = 1$.

- $\texttt{init}(S)$: Using repeated squaring, we construct the string 0^u by inserting $w = 0$ and concatenating w with itself $\log u$ times. Note that 0^u will remain in the data structure and can be used by other operations. We compute x_S by updating 0^u using $\texttt{update}(x_S, i, 1)$ for all indices $i \in S$. Since the repeated squaring takes time $O(\log^2 u)$ and updating the elements takes time $O(|S| \cdot \log u)$, the `init` operation runs in time $O(\log^2 u + |S| \cdot \log u)$.

- $\texttt{query}(i)$: As the original data structure already provides a query operation, we use $\texttt{query}(x_S, i)$ that returns $x_S[i]$ in time $O(\log u)$.

- $\texttt{sum}(p)$: We implement `sum` in three steps. First, we will compute the string x_{S+p} that represents the set $S + p$. Observe that x_{S+p} is obtained by shifting x_S by p positions to the right. Thus, we extend x_S using $\texttt{concat}(0^p, x_S)$ where the string 0^p is split off the precomputed string 0^u using $\texttt{split}(0^u, p-1)$. Then we trim it down to length u with $\texttt{split}(0^p x_S, u-1)$. (Note that due to the assumption $\max(S+p) < u$ we only split off zeros in this step.)

  Second, note that the desired string $x_{S \cup (S+p)}$ is the result of the bitwise OR of x_S and x_{S+p}. We compute the set $D = \{i \in [u] : x_S[i] \neq x_{S+p}[i]\}$ that contains all indices at which x_S and x_{S+p} differ from each other. To this end, starting with $i \leftarrow 0$, we will repeat the following process as long as $i < u$: Compute $\ell \leftarrow \texttt{LCE}(x_S, x_{S+p}, i, i)$ to determine the next index $i + \ell$ at which both strings differ, insert $D \leftarrow D \cup \{i + \ell\}$ and update $i \leftarrow i + \ell + 1$.

  As the third and last step, we compute $x_{S \cup (S+p)}$ by updating x_S using $\texttt{update}(x_S, i, 1)$ for all indices $i \in D$.

  Creating the shifted string x_{S+p} takes time $O(\log u)$. Both the construction of set D and computing the string $x_{S \cup (S+p)}$ require $|(S+p) \setminus S| + |S \setminus (S+p)|$ many operations that each run in time $O(\log u)$. Since $|S + p| = |S|$, we have

  $$|(S+p) \setminus S| = |S+p| - |(S+p) \cap S| = |S| - |(S+p) \cap S| = |S \setminus (S+p)|$$

  and therefore $|(S+p) \setminus S| + |S \setminus (S+p)| = 2 \cdot |(S+p) \setminus S|$. In summary, the `sum` operation takes time $O(|(S+p) \setminus S| \cdot \log u)$.

- `cap`(d): In order to set $x_S[i] = 0$ for all $i \in (d\,..\,u)$, we separate the substring $x_S[0\,..\,d]$ with `split`(x_S, d) and split the substring 0^{u-d-1} off the precomputed string 0^u using `split`$(0^u, u-d-2)$. Then x_S is assembled using `concat`$(x_S[0\,..\,d], 0^{u-d-1})$. All three operations take time $O(\log u)$.

Following Lemma 3.3, the deterministic running times can be obtained by worsening all operations by a factor $\log^{o(1)} u$. □

3.2 Obtaining an Optimal Schedule

In the previous sections we have argued that the optimal value OPT (i.e., the maximum total processing of a feasible schedule) can be computed in near-linear time $\widetilde{O}(P)$. In this section we explain how the actual optimal schedule can be computed by a deterministic post-processing routine in time $O(n)$.

The idea is, as is standard for dynamic programming algorithms, to trace through the sets $S_0, \ldots, S_n$ in reverse order. To make this traversal efficient, we slightly modify our algorithm to additionally compute an array $A[0\,..\,P]$ such that $A[s] = \min\{j \in [n] : s \in S_{j+1}\}$. Intuitively, $A[s]$ stores the smallest job j such that there exists a feasible schedule with total processing time s that contains j and a subset of the jobs $\{0, \ldots, j-1\}$. It is easy to appropriately maintain the array A whenever an element is inserted into S'_{j+1} without worsening the asymptotic running time. Then, in order to compute an optimal schedule, we apply the following algorithm: We initialize $J \leftarrow \emptyset$ and $s \leftarrow \text{OPT}$. We repeatedly retrieve the next job $j \leftarrow A[s]$ and update $J \leftarrow J \cup \{j\}$ and $s \leftarrow s - p_j$, until $s = 0$. In each step, we identify a job j that is contained in the optimal schedule, and thus J is an optimal schedule once the process has terminated. In fact, the same idea can be used to retrieve a feasible schedule for any given processing time $s \in S_n$.

4 Generalization to Multiple Machines

In this section, we show that our algorithm for $1||\sum p_j U_j$ can be extended to $P_m||\sum p_j U_j$. Since it follows the same approach as the single machine algorithm, we will keep this section short and concise. For more details refer to Section 3.

Let $e_0, \ldots, e_{m-1}$ denote the standard unit vectors of $\mathbb{Z}^m$, then we recursively define the sets $S_0, \ldots, S_n \subseteq [0, P]^m$ as follows:

$$\begin{aligned} S_0 &= \{0\}, \\ S'_{j+1} &= S_j + \{0, p_j \cdot e_0, \ldots, p_j \cdot e_{m-1}\} && (j \in [n]), \\ S_{j+1} &= S'_{j+1} \cap [0, d_j]^m && (j \in [n]). \end{aligned}$$

As before, the optimal value is the maximum entry in S_n:

Observation 4.1 (Lawler and Moore [31]). There is a feasible schedule of total processing time t if and only if there is some $s \in S_n$ with $s_0 + \cdots + s_{m-1} = t$.

The crucial difference to before is that here all $s \in S_j$ are vectors where their i-th entry corresponds to the i-th machine. Because each job is either scheduled on exactly one machine or not at all, we consider all scheduling possibilities of

job j with $S_j + \{0, p_j \cdot e_0, \ldots, p_j \cdot e_{m-1}\}$. As our goal is again to bound the total number of insertions, see the following lemma:

Lemma 4.1 (Generalized Bounded Insertions). It holds that

$$\sum_{j \in [n]} \left| S'_{j+1} \setminus S_j \right| \leq (m+1) \cdot (P+1)^m.$$

Proof. In the following, we consider two parts of the sum separately:

$$\sum_{j \in [n]} \left| (S'_{j+1} \setminus S_j) \right| = \sum_{j \in [n]} \left| (S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]^m \right| + \sum_{j \in [n]} \left| (S'_{j+1} \setminus S_j) \setminus [0\,..\,d_j]^m \right|.$$

In other words, in analogy to Lemma 3.2, we first count the number of elements that are irrevocably inserted into $S_{j+1}, \ldots, S_n$ in the j-th step. Second, we count the number of elements that are inserted into S'_{j+1} and instantly deleted in S_{j+1}.

We bound the first sum with the following observation. For any $x \in [0\,..\,P]^m$, it holds that if $x \in (S'_{j+1} \setminus S_j)$ and $x \in [0\,..\,d_j]$, then $x \in S_{j+1}, \ldots, S_n$. This follows directly from the assumption that $d_j \leq d_{j+1}, \ldots, d_{n-1}$. Therefore, it holds that

$$\left| \{ j \in [n] : x \in (S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]^m \} \right| \leq 1,$$

for all $x \in [0\,..\,P]^m$, and thus

$$\sum_{j \in [n]} \left| (S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]^m \right| = \sum_{x \in [0\,..\,P]^m} \left| \{ j \in [n] : x \in (S'_{j+1} \setminus S_j) \cap [0\,..\,d_j]^m \} \right| \leq (P+1)^m.$$

For the second sum, we bound $|(S'_{j+1} \setminus S_j) \setminus [0\,..\,d_j]^m| \leq |S'_{j+1} \setminus [0\,..\,d_j]^m|$. Using the fact that $S_j \subseteq [0\,..\,d_{j-1}]^m$, we have that

$$\begin{aligned} S'_{j+1} &= S_j + \{0, p_j \cdot e_0, \ldots, p_j \cdot e_{m-1}\} \\ &\subseteq [0\,..\,d_{j-1}]^m + \{0, p_j \cdot e_0, \ldots, p_j \cdot e_{m-1}\} =: V_{j+1}. \end{aligned}$$

As each job j is scheduled on exactly one machine, we observe that V_{j+1} is the set of vectors where all entries are in $[0\,..\,d_{j-1}]$, except for possibly one entry that is in $[0\,..\,d_{j-1} + p_j]$. Hence, $V_{j+1} \setminus [0\,..\,d_j]^m$ is the set of vectors where all entries are in $[0\,..\,d_{j-1}]$, except for exactly one entry that is in $(d_j\,..\,d_{j-1} + p_j]$. Next, we bound the size of $V_{j+1} \setminus [0\,..\,d_j]^m$: There are m options for the index of the special entry, there are $d_{j-1} + p_j - d_j$ options for the value of the special entry, and finally there are $(P+1)^{m-1}$ options for the other $m-1$ entries. Thus,

$$\begin{aligned} \left| S'_{j+1} \setminus [0\,..\,d_j]^m \right| &\leq \left| V_{j+1} \setminus [0\,..\,d_j]^m \right| \\ &\leq m \cdot (P+1)^{m-1} \cdot (d_{j-1} + p_j - d_j) \\ &\leq m \cdot (P+1)^{m-1} \cdot p_j, \end{aligned}$$

where the final inequality follows from the assumption that $d_{j-1} \leq d_j$. Consequently, the second sum is bounded by

$$\sum_{j \in [n]} \left| S'_{j+1} \setminus [0\,..\,d_j]^m \right| \leq m \cdot (P+1)^{m-1} \cdot \sum_{j \in [n]} p_j \leq m \cdot (P+1)^m.$$

Combining the bounds for both sums yields the overall bound. □

Analogous to Section 3, we use the generalized sum-cap data structure to efficiently maintain the generalized sets $S_0, \ldots, S_n$.

Lemma 4.2 (Generalized Sum-Cap Data Structure). There is a randomized data structure maintaining a set $S \subseteq [u]^m$ that supports the following operations:

- `init`(S): Initializes the data structure to the given set $S \subseteq [u]^m$. Runs in time $O(\log^2 u + |S| \cdot \log u)$.
- `query`(s): Given $s \in [u]^m$, tests whether $s \in S$. Runs in time $O(\log u)$.
- `sum`(T): Given $T \subseteq [u]^m$, updates $S \leftarrow S + T$. Runs in time $O(|T| \cdot |(S+T) \setminus S| \cdot \log u)$ (where S is as before the operation).
- `cap`(d): Given $d \in [u]$, updates $S \leftarrow S \cap [d]^m$. Runs in time $O(u^{m-1} \cdot \log u)$.

If at any point during the execution an element $s \notin [u]^m$ is attempted to be inserted, the data structure becomes invalid. Moreover, the data structure can be made deterministic at the cost of worsening all operations by a factor $\log^{o(1)} u$.

Proof. Let $U = u^m$. Let $\phi : [u]^m \to [U]$ be the bijection defined by $\phi(s) = \sum_{i \in [m]} s_i u^i$. We extend the definition to sets $S \subseteq [u]^m$ via $\phi(S) = \{\phi(s) : s \in S\}$. We maintain the set $S \subseteq [u]^m$ as the indicator string of $\phi(S)$, namely $x_{\phi(S)} \in \{0,1\}^U$, such that $i \in S$ if and only if $x_{\phi(S)}[\phi(i)] = 1$. In other words, we store for each $s \in [u]^m$, listed in lexicographical order, whether $s \in S$.

- `init`(S): The string 0^U is constructed using repeated squaring by inserting $x = 0$ and concatenating x with itself $m \log u$ times. It will remain in the data structure available to other operations. Repeatedly using `update`$(\cdot, \phi(s), 1)$ for all entries $s \in S$, we update the string 0^U to obtain $x_{\phi(S)}$. As repeated squaring takes time $O(\log^2 u)$ and updating the elements takes time $O(|S| \cdot \log u)$, the `init` operation runs in time $O(\log^2 u + |S| \cdot \log u)$.
- `query`(s): Using `query`$(x_{\phi(S)}, s)$ from the string data structure allows us to return $x_{\phi(S)}[s]$ in time $O(\log u)$.
- `sum`(T): We can assume without loss of generality that $0 \in T$ (as otherwise we can simply shift T and $x_{\phi(S)}$ appropriately). Fix an arbitrary nonzero element $p \in T$. Analogously to Lemma 3.1, we first show how to compute the set $(S+p) \setminus S$ in output-sensitive time. The string $x_{\phi(S+p)}$ representing the set $S+p$ can be computed using the two following facts. If $x, y, x+y \in [u]^m$, then it holds that $\phi(x+y) = \phi(x) + \phi(y)$. Further, if $p \in [u]^m$ and $S, S+p \subseteq [u]^m$, then $\phi(S+p) = \phi(S) + \phi(p)$. Therefore, $x_{\phi(S+p)}$ is $x_{\phi(S)}$ up to a shift of $\phi(p)$, and can be obtained by `split`$(\texttt{concat}(0^{\phi(p)}, x_{\phi(S)}), U-1)$.

  Similarly to Lemma 3.1, repeatedly using `LCE` queries allows us to first compute the set $\{i \subseteq [U] : x_{\phi(S)}[i] \neq x_{\phi(S+p)}[i]\}$ and then read off the symmetric difference of S and $S+p$, denoted by $D_p = (S \setminus (S+p)) \cup$

$((S+p) \setminus S)$. We repeat the same for all other nonzero elements $p \in T$. Let $D = \bigcup_{p \in T \setminus \{0\}} D_p$, then we have $D \supseteq (S+T) \setminus S$. Thus, we can update the indicator string $x_{\phi(S)}$ by calling $\texttt{update}(\cdot, \phi(s), 1)$ for all $s \in D$.

In order to bound the running time, we bound the size of the sets D_p and D. Using that $|S \setminus (S+p)| = |(S+p) \setminus S|$, we have that $|D_p| \leq 2|(S+p) \setminus S| \leq 2|(S+T) \setminus S|$, and therefore $|D| \leq 2|T| \cdot |(S+T) \setminus S|$. In total, we used $O(|D|)$ data structure operations, leading to a running time of $O(|T| \cdot |(S+T) \setminus S| \cdot \log u)$.

- $\texttt{cap}(d)$: We delete all vectors $s \in S$ with at least one entry that is larger than d as follows. We enumerate all $(m-1)$-tuples $(s_1, \ldots, s_{m-1}) \in [u]^{m-1}$. Recall that the vectors are stored in lexicographical order. Thus, the set of vectors $(s_0, s_1, \ldots, s_{m-1})$ where s_0 ranges over $[u]$ and $s_1, \ldots, s_{m-1}$ are fixed, is represented by a length-u substring of $x_{\phi(S)}$. Specifically, the substring $x_{\phi(S)}[\phi(0, s_1, \ldots, s_{m-1}) .. \phi(u-1, s_1, \ldots, s_{m-1})]$. We distinguish two cases: If $\max_{i=1}^{m-1} s_i > d$, then the entire substring is replaced with 0^u. Otherwise, we retain its length-$(d+1)$ prefix and replace its suffix is by 0^{u-d-1}. It takes $O(u^{m-1})$ $\texttt{concat}$ and $\texttt{split}$ operations to perform these modifications, running in total time $O(u^{m-1} \cdot \log u)$.

Again, following Lemma 3.3 the deterministic running time of the operations differs by replacing $\log u$ with $\log^{1+o(1)} u$. □

Based on Lemma 4.1 and 4.2, we show the following generalization of our main result.

Theorem 1.2. (Restated) The $P_m || \sum p_j U_j$ problem can be solved in randomized time $O(P^m \log P)$ and in deterministic time $O(P^m \log^{1+o(1)} P)$.

Proof. Analogous to Theorem 1.1, we use the algorithm: We initialize the data structure from Lemma 4.2 (used with $u = P+1$) with $\texttt{init}(S_0)$. For all $j \leftarrow 0, \ldots, n-1$, we repeatedly use the operations $\texttt{sum}(\{0, p_j \cdot e_0, \ldots, p_j \cdot e_{m-1}\})$ and $\texttt{cap}(d_j)$ to compute the sets S'_{j+1} and S_{j+1}. We return the optimal value contained in S_n as described in Theorem 1.1. Our algorithm is correct due to Observation 4.1. Finally, using Lemmas 4.1 and 4.2 and the assumption $n \leq P$, we can bound the dominant term of the running time by

$$O\left(\sum_{j \in [n]} (m \cdot |S'_{j+1} \setminus S_j| \cdot \log P + P^{m-1} \log P)\right) = O(P^m \log P).$$

(Recall that m is constant.) Thus, we obtain a randomized running time of $O(P^m \log P)$, and similarly a deterministic running time of $O(P^m \log^{1+o(1)} P)$. □

References

[1] Amir Abboud, Karl Bringmann, Danny Hermelin, and Dvir Shabtay. Seth-based lower bounds for subset sum and bicriteria path. ACM Trans. Algorithms, 18(1):6:1–6:22, 2022. doi: 10.1145/3450524. URL https://doi.org/10.1145/3450524.

[2] Stephen Alstrup, Gerth Stølting Brodal, and Theis Rauhe. Pattern matching in dynamic texts. In David B. Shmoys, editor, 11th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2000), pages 819–828. ACM/SIAM, 2000. URL http://dl.acm.org/citation.cfm?id=338219.338645.

[3] Antonis Antonopoulos, Aris Pagourtzis, Stavros Petsalakis, and Manolis Vasilakis. Faster algorithms for k-subset sum and variations. J. Comb. Optim., 45(1):24, 2023. doi: 10.1007/S10878-022-00928-0. URL https://doi.org/10.1007/s10878-022-00928-0.

[4] Kyriakos Axiotis and Christos Tzamos. Capacitated dynamic programming: Faster knapsack and graph algorithms. In Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, editors, 46th International Colloquium on Automata, Languages, and Programming (ICALP 2019), volume 132 of LIPIcs, pages 19:1–19:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. doi: 10.4230/LIPICS.ICALP.2019.19. URL https://doi.org/10.4230/LIPIcs.ICALP.2019.19.

[5] Kyriakos Axiotis, Arturs Backurs, Ce Jin, Christos Tzamos, and Hongxun Wu. Fast modular subset sum using linear sketching. In Timothy M. Chan, editor, 30th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2019), pages 58–69. SIAM, 2019. doi: 10.1137/1.9781611975482.4. URL https://doi.org/10.1137/1.9781611975482.4.

[6] Kyriakos Axiotis, Arturs Backurs, Karl Bringmann, Ce Jin, Vasileios Nakos, Christos Tzamos, and Hongxun Wu. Fast and simple modular subset sum. In Hung Viet Le and Valerie King, editors, 4th Symposium on Simplicity in Algorithms (SOSA 2021), pages 57–67. SIAM, 2021. doi: 10.1137/1.9781611976496.6. URL https://doi.org/10.1137/1.9781611976496.6.

[7] MohammadHossein Bateni, MohammadTaghi Hajiaghayi, Saeed Seddighin, and Cliff Stein. Fast algorithms for knapsack via convolution and prediction. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, 50th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2018), pages 1269–1282. ACM, 2018. doi: 10.1145/3188745.3188876. URL https://doi.org/10.1145/3188745.3188876.

[8] Karl Bringmann. A near-linear pseudopolynomial time algorithm for subset sum. In Philip N. Klein, editor, 28th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2017), pages 1073–1084. SIAM, 2017. doi: 10.1137/1.9781611974782.69. URL https://doi.org/10.1137/1.9781611974782.69.

[9] Karl Bringmann. Knapsack with small items in near-quadratic time. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024, pages 259–270. ACM, 2024. doi: 10.1145/3618260.3649719. URL https://doi.org/10.1145/3618260.3649719.

[10] Karl Bringmann and Alejandro Cassis. Faster knapsack algorithms via bounded monotone min-plus-convolution. In Mikolaj Bojanczyk, Emanuela

Merelli, and David P. Woodruff, editors, 49th International Colloquium on Automata, Languages, and Programming (ICALP 2022), volume 229 of LIPIcs, pages 31:1–31:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi: 10.4230/LIPICS.ICALP.2022.31. URL https://doi.org/10.4230/LIPIcs.ICALP.2022.31.

[11] Karl Bringmann and Alejandro Cassis. Faster 0-1-knapsack via near-convex min-plus-convolution. In Inge Li Gørtz, Martin Farach-Colton, Simon J. Puglisi, and Grzegorz Herman, editors, 31st Annual European Symposium on Algorithms (ESA 2023), volume 274 of LIPIcs, pages 24:1–24:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPICS.ESA.2023.24. URL https://doi.org/10.4230/LIPIcs.ESA.2023.24.

[12] Karl Bringmann and Philip Wellnitz. On near-linear-time algorithms for dense subset sum. In Dániel Marx, editor, 32nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2021), pages 1777–1796. SIAM, 2021. doi: 10.1137/1.9781611976465.107. URL https://doi.org/10.1137/1.9781611976465.107.

[13] Karl Bringmann, Nick Fischer, Danny Hermelin, Dvir Shabtay, and Philip Wellnitz. Faster minimization of tardy processing time on a single machine. Algorithmica, 84(5):1341–1356, 2022. doi: 10.1007/S00453-022-00928-W. URL https://doi.org/10.1007/s00453-022-00928-w.

[14] Jean Cardinal and John Iacono. Modular subset sum, dynamic strings, and zero-sum sets. In Hung Viet Le and Valerie King, editors, 4th Symposium on Simplicity in Algorithms (SOSA 2021), pages 45–56. SIAM, 2021. doi: 10.1137/1.9781611976496.5. URL https://doi.org/10.1137/1.9781611976496.5.

[15] Timothy M. Chan and Qizheng He. More on change-making and related problems. J. Comput. Syst. Sci., 124:159–169, 2022. doi: 10.1016/J.JCSS.2021.09.005. URL https://doi.org/10.1016/j.jcss.2021.09.005.

[16] Lin Chen, Jiayi Lian, Yuchen Mao, and Guochuan Zhang. Faster algorithms for bounded knapsack and bounded subset sum via fine-grained proximity results. In 35th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2024), pages 4828–4848. SIAM, 2024. doi: 10.1137/1.9781611977912.171. URL https://doi.org/10.1137/1.9781611977912.171.

[17] Marek Cygan, Holger Dell, Daniel Lokshtanov, Dániel Marx, Jesper Nederlof, Yoshio Okamoto, Ramamohan Paturi, Saket Saurabh, and Magnus Wahlström. On problems as hard as CNF-SAT. ACM Trans. Algorithms, 12(3):41:1–41:24, 2016. doi: 10.1145/2925416. URL https://doi.org/10.1145/2925416.

[18] Marek Cygan, Marcin Mucha, Karol Wegrzycki, and Michal Wlodarczyk. On problems equivalent to (min, +)-convolution. ACM Trans. Algorithms, 15(1):14:1–14:25, 2019. doi: 10.1145/3293465. URL https://doi.org/10.1145/3293465.

[19] Mingyang Deng, Xiao Mao, and Ziqian Zhong. On problems related to unbounded SubsetSum: A unified combinatorial approach. In Nikhil Bansal and Viswanath Nagarajan, editors, 32nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2021), pages 2980–2990. SIAM, 2023. doi: 10.1137/1.9781611977554.CH114. URL https://doi.org/10.1137/1.9781611977554.ch114.

[20] Friedrich Eisenbrand and Robert Weismantel. Proximity results and faster algorithms for integer programming using the Steinitz lemma. ACM Trans. Algorithms, 16(1):5:1–5:14, 2020. doi: 10.1145/3340322. URL https://doi.org/10.1145/3340322.

[21] Pawel Gawrychowski, Adam Karczmarz, Tomasz Kociumaka, Jakub Lacki, and Piotr Sankowski. Optimal dynamic strings. In Artur Czumaj, editor, 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018), pages 1509–1528. SIAM, 2018. doi: 10.1137/1.9781611975031.99. URL https://doi.org/10.1137/1.9781611975031.99.

[22] Ronald L. Graham, Eugene L. Lawler, Jan K. Lenstra, and Alexander H. G. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. Annals of Discrete Mathematics, 5: 287–326, 1979. ISSN 0167-5060. doi: 10.1016/S0167-5060(08)70356-X.

[23] Danny Hermelin, Hendrik Molter, and Dvir Shabtay. Minimizing the weighted number of tardy jobs via (max,+)-convolutions. INFORMS J. Comput., 36(3):836–848, 2024. doi: 10.1287/IJOC.2022.0307. URL https://doi.org/10.1287/ijoc.2022.0307.

[24] Klaus Jansen and Lars Rohwedder. On integer programming, discrepancy, and convolution. Math. Oper. Res., 48(3):1481–1495, 2023. doi: 10.1287/MOOR.2022.1308. URL https://doi.org/10.1287/moor.2022.1308.

[25] Ce Jin. 0-1 knapsack in nearly quadratic time. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024, pages 271–282. ACM, 2024. doi: 10.1145/3618260.3649618. URL https://doi.org/10.1145/3618260.3649618.

[26] Ce Jin and Hongxun Wu. A simple near-linear pseudopolynomial time randomized algorithm for subset sum. In Jeremy T. Fineman and Michael Mitzenmacher, editors, 2nd Symposium on Simplicity in Algorithms (SOSA 2019), volume 69 of OASIcs, pages 17:1–17:6. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. doi: 10.4230/OASICS.SOSA.2019.17. URL https://doi.org/10.4230/OASIcs.SOSA.2019.17.

[27] Dominik Kempa and Tomasz Kociumaka. Dynamic suffix array with polylogarithmic queries and updates. In Stefano Leonardi and Anupam Gupta, editors, 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022), pages 1657–1670. ACM, 2022. doi: 10.1145/3519935.3520061. URL https://doi.org/10.1145/3519935.3520061.

[28] Kim-Manuel Klein, Adam Polak, and Lars Rohwedder. On minimizing tardy processing time, max-min skewed convolution, and triangular structured ILPs. In Nikhil Bansal and Viswanath Nagarajan, editors, 34th

Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2023), pages 2947–2960. SIAM, 2023. doi: 10.1137/1.9781611977554.CH112. URL https://doi.org/10.1137/1.9781611977554.ch112.

[29] Konstantinos Koiliaris and Chao Xu. Faster pseudopolynomial time algorithms for subset sum. ACM Trans. Algorithms, 15(3):40:1–40:20, 2019. doi: 10.1145/3329863. URL https://doi.org/10.1145/3329863.

[30] Marvin Künnemann, Ramamohan Paturi, and Stefan Schneider. On the fine-grained complexity of one-dimensional dynamic programming. In Ioannis Chatzigiannakis, Piotr Indyk, Fabian Kuhn, and Anca Muscholl, editors, 44th International Colloquium on Automata, Languages, and Programming (ICALP 2017), volume 80 of LIPIcs, pages 21:1–21:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi: 10.4230/LIPICS.ICALP.2017.21. URL https://doi.org/10.4230/LIPIcs.ICALP.2017.21.

[31] Eugene L. Lawler and J. Michael Moore. A functional equation and its application to resource allocation and sequencing problems. Management Science, 16(1):77–84, 1969. doi: 10.1287/mnsc.16.1.77. URL https://doi.org/10.1287/mnsc.16.1.77.

[32] Kurt Mehlhorn, R. Sundar, and Christian Uhrig. Maintaining dynamic sequences under equality tests in polylogarithmic time. Algorithmica, 17 (2):183–198, 1997. doi: 10.1007/BF02522825. URL https://doi.org/10.1007/BF02522825.

[33] Adam Polak, Lars Rohwedder, and Karol Wegrzycki. Knapsack and subset sum with small items. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, 48th International Colloquium on Automata, Languages, and Programming (ICALP 2021), volume 198 of LIPIcs, pages 106:1–106:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi: 10.4230/LIPICS.ICALP.2021.106. URL https://doi.org/10.4230/LIPIcs.ICALP.2021.106.

[34] Krzysztof Potępa. Faster deterministic modular subset sum. In Petra Mutzel, Rasmus Pagh, and Grzegorz Herman, editors, 29th Annual European Symposium on Algorithms, (ESA 2021), volume 204 of LIPIcs, pages 76:1–76:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi: 10.4230/LIPICS.ESA.2021.76. URL https://doi.org/10.4230/LIPIcs.ESA.2021.76.

[35] William W. Pugh and Tim Teitelbaum. Incremental computation via function caching. In 16th Annual ACM Symposium on Principles of Programming Languages (POPL 1989), pages 315–328. ACM Press, 1989. doi: 10.1145/75277.75305. URL https://doi.org/10.1145/75277.75305.

[36] Baruch Schieber and Pranav Sitaraman. Quick minimization of tardy processing time on a single machine. In Pat Morin and Subhash Suri, editors, 18th International Symposium on Algorithms and Data Structures (WADS 2023), volume 14079 of Lecture Notes in Computer Science, pages 637–643. Springer, 2023. doi: 10.1007/978-3-031-38906-1_42. URL https://doi.org/10.1007/978-3-031-38906-1_42.

[37] Rajamani Sundar and Robert Endre Tarjan. Unique binary-search-tree representations and equality testing of sets and sequences. SIAM J. Comput., 23(1):24–44, 1994. doi: 10.1137/S0097539790189733. URL https://doi.org/10.1137/S0097539790189733.

[38] Arie Tamir. New pseudopolynomial complexity bounds for the bounded and other integer knapsack related problems. Oper. Res. Lett., 37(5):303–306, 2009. doi: 10.1016/J.ORL.2009.05.003. URL https://doi.org/10.1016/j.orl.2009.05.003.